Comprehension and Application of Design Patterns by Novice Software Engineers

An Empirical Study of Undergraduate Software Engineering and Computer Science Students

Jonathan W. Lartigue
Auburn University
Auburn, Alabama
lartijw@auburn.edu

Richard Chapman
Auburn University
Auburn, Alabama
chapmro@auburn.edu

ABSTRACT

Although there has been a large body of work cataloguing design patterns since their introduction, there is a limited amount of detailed, empirical evidence on pattern use and application. Those studies that have collected experimental data generally focus on experienced, professional software engineers or graduate-level computer science and software engineering students.

Although the value of design pattens in general is still widely debated, many experts have concluded that the use of design patterns is beneficial for experienced software engineers and architects. But it is still unclear if the benefits of design patterns translate equally to young, inexperienced software engineers.

To assess this, we conducted a controlled experiment to evaluate the comparative performance in targeted tasks of novice software engineers, which are represented by software engineering undergraduate students about to earn a bachelors degree in an ABET-accredited computer science or software engineering program. We assessed the ability of subjects to recognize, comprehend, and refactor software containing a number of design patterns. We also collected subjective data measuring the subjects' preferences for or against pattern use.

Although experiment results are mixed, depending on the complexity of the pattern involved, we observe that novice software engineers can recognize and understand software containing some design patterns, but that benefits of pattern use, in terms of refactoring time, are dependent on the complexity of the pattern. We conclude that, while simpler patterns show benefits, more complex design patterns may be an impediment for novice developers.

CCS CONCEPTS

• **Software and its engineering → Design patterns**; *Reusability*; Software design techniques; • **Social and professional topics →** *Software engineering education*; *Computer science education*;

KEYWORDS

Design Patterns, Reusability, Education, Programming Pedagogy, Novice

ACM Reference Format:
Jonathan W. Lartigue and Richard Chapman. 2018. Comprehension and Application of Design Patterns by Novice Software Engineers: An Empirical Study of Undergraduate Software Engineering and Computer Science Students. In *ACM SE '18: ACM SE '18: Southeast Conference, March 29–31, 2018, Richmond, KY, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3190645.3190686

ACM SE '18, March 29–31, 2018, Richmond, KY, USA

ACM ISBN 978-1-4503-5696-1/18/03…$15.00
https://doi.org/10.1145/3190645.3190686

1 INTRODUCTION

Design patterns are high-level abstractions of specific solutions to common software engineering tasks or problems. They are, in the loosest sense, a generic recipe that developers can adapt and tailor to their needs when solving specific software tasks that are described by a particular pattern. A primary goal of design patterns is to allow programmers to spend less time on "mundane, repetitive parts of" programming and more on the problems that are unique to each program. [6]

Design patterns are intended to be applied over and over, in any sufficiently high-level language or framework, and in such a way that use of the pattern adds value in the form of a consistent implementation recognizable and understandable by sufficiently trained software engineers, thus reducing refactoring costs in later development cycles. [17]

The question of whether younger, less-experienced engineers can benefit equally from these patterns has largely been left unaddressed, but some claim that "patterns help a novice perform more like an expert" [17] and that "(p)atterns help you build on the collective experience of *skilled* software engineers" (emphasis added). [9]

Several studies have been done on the ability of graduate students and experienced software engineers to grasp and utilize design patterns effectively [31, 36], but few, if any, studies have focused on either undergraduate students or newly graduated novice software engineers.

We chose to investigate the ability of young software engineers to recognize, understand, and employ several well-known design patterns in a controlled experiment involving the completion of several software refactoring tasks. Subjects for the study were recruited from students enrolled in a junior-level software modeling and design course in an ABET-accredited computer science or ABET-accredited software engineering program. Subjects were

given a short introduction to design patterns and then asked to perform several software maintenance tasks, some of which involved design patterns and some of which did not.

2 DESIGN PATTERNS OVERVIEW

The origin of design patterns in engineering, as well as computer science and software engineering specifically, is often traced to the work of architect Christopher Alexander, and others, who developed a series of generalized designs for dozens of architectural needs that could be quickly adapted and implemented in specific projects. The insight of this concept was that many recurring architectural needs could be formalized, described, and solved at a general level such that specific implementations could be repeatedly and reliably generated from these patterns. [1]

Alexander proposed the term "pattern language" to describe a methodology, or grammar, for formally describing a set of common problems of design and implementation of solutions to those problems. These were quickly recognized and adapted by many engineering disciplines and are especially applicable to the field of software engineering, which is replete with computational tasks that have been implemented and re-implemented on countless computing platforms in scores of languages. [14] Early work in adapting design patterns to computer science was led by, among others, Kent Beck [5, 7, 8] and Jim Coplien. [13]

Early design patterns were poorly understood and difficult to communicate until a formal definition of software design patterns, a framework for how to document them, and a catalog of common patterns and their applications was published by the so-called "Gang of Four." [17] This not only offered a language and framework for describing design patterns, their intent, structure, and application, but also offered a catalog of 23 design patterns that addressed frequent design problems. Gamma, et al., envisioned their catalog of patterns "as a new mechanism for expressing object-oriented design experience" and "a reusable base of experience for building reusable software." [16–18]

This implies that these patterns are cataloged by and intended for use by other developers who possess similar experience and skill. Gamma, et al., state that "(g)ood designers, it appears, rely on large amounts of design experience, and this experience is just as important as the notations for recording designs and the rules for using those notations." They go further: "a software architect who is familiar with a good set of design structures can apply them immediately to design problems without having to rediscover them." [17]

Design patterns have significantly influenced the manner in which object-oriented software is designed, but research is mixed on whether specific design patterns, or even patterns as a whole, improve the quality of software systems. [11, 20]

However, the use of patterns may obfuscate what could otherwise be accomplished more simply, even if less conducive to future refactoring. "Patterns make it easy to make a system complex," states Gamma. "They achieve flexibility and variability by introducing levels of indirection, which can complicate a design." [15]

Johnson suggests that computer scientists sometimes have a tendency to create new methods for classifying or solving CS problems — a tendency, he states, to create new 'theories' that can solve problems rather than simply solving the problems themselves. [2] Beck implies that this can lead to trouble when attempted by the relatively inexperienced: "(I)f you try to build a theory without having enough experience in the problem, you are unlikely to find a good solution. Moreover, much of the information in a design is not derived from first principles, but obtained by experience." [7]

2.1 Design Patterns Studies

Despite the large number of publications regarding design patterns, there has been an extremely small amount of detailed, empirical evidence on their use and application since their introduction. [38] One extensive literature review found that of 611 candidate papers on patterns, only 11 of these presented any formal experimental studies of patterns that produced empirical evidence. [21]

"The large number of papers and books published about patterns supports the view that the concept of the design pattern is valued by many experienced developers," state Zhang and Budgen. "However, the available literature on patterns appears to be largely in the form of 'advocacy' or 'experience,' and much of it is also focused upon identifying and documenting patterns rather than reporting and analyzing experience with using them." [38] They found that 15 of the 23 original "Gang of Four" design patterns had been subjected to some form of empirical evaluations, but that six of these were only addressed in a single study, and conclude that "design patterns have not been subjected to other than limited empirical evaluation, and that much of this has also only been studying patterns indirectly." [38]

Among the more thorough quantitative experiments on the usefulness of design patterns were a series of studies conducted by Lutz Prechelt and colleagues beginning in the late 1990s and continuing in the early 2000s. [25–27, 29]

Prechelt, et al., noted that although "subjective" evidence existed at the time for pattern use, "no rigorous test has yet been presented that design patterns are useful." [27] In one of the first such quantitative experiments on design pattern use, Prechelt conducted a controlled study into the benefits of well-documented design patterns on both task completion speed and also accuracy, seeking to prove that: maintenance tasks can be completed more quickly, on average, with pattern documentation in code than without; and, fewer errors will be committed, on average, in such tasks with pattern documentation than without. [24]

Their experiment included 74 subjects with extensive previous programming experience, and focused on the ability of these subjects to perform software maintenance tasks on programs that incorporated patterns. The subjects were students in the equivalent of either a computer science bachelors or masters program, had an average of 7.52 years of programming experience (with 75 percent having five or more years' experience), and 64 already held the equivalent of a bachelor's degree in the field. [24, 27]

Subjects were selected from those who had recently completed a course that covered Java, the Abstract Window Toolkit, and specifically included five common design patterns: Composite, Observer,

Strategy, Template Method, and Visitor. [27] The experiment, conducted entirely on paper, involved two Java programs of 362 and 565 lines of code incorporating several fundamental design patterns, such as Composite, Visitor, Observer and Template Method patterns. Tasks were designed to take, on average, one hour and centered around application domains simple enough to be understood readily by subjects and only focused on the five design patterns introduced in the course. Only small amounts of new or modified code needed to be written by subjects.

Prechelt, et al., concluded that documentation of software design patterns significantly decreased the amount of time required to perform the indicated maintenance tasks. [27] However, the authors note that the experiment was "conservative" in that the subjects had recently received patterns training and knew they were participating in an study specifically focusing on design patterns. A very interesting observation is that subjects in the experimental group working on maintenance tasks involving the Visitor pattern required *longer* than the control group to complete the maintenance task. [27]

Prechelt, et al., repeated the original experiment on the value of design pattern documentation a year later to add an additional subject trial, this time with running code. At the same time, they attempted to answer the question of whether patterns were beneficial, compared to alternative designs, and added a separate experiment to assess the value of patterns in maintenance tasks.

They found that design patterns can be beneficial, even if a simpler solution is available, but also that patterns can be applied excessively or incorrectly, resulting in less desirable code. [25] They concluded that "unless there is a clear reason to prefer the simpler solution, it is probably wise to choose the flexibility provided by the design pattern because unexpected new requirements often appear." [28]

Other research has shown that in large commercial projects, the application of design patterns can become uncontrolled and, especially among engineers less experienced with patterns, their use in an "exploratory way" can lead to negative results and maintenance problems. Furthermore, because design patterns can be tightly coupled to other elements of the system, their removal is sometimes "economically not viable." [37]

Chatzigeorgiou, et al., expanded upon the earlier work by Prechelt, et al., in a 2007 study to assess postgraduate students' abilities to comprehend design patterns by comparing the work product from two software projects, one with and one without design patterns. Subjects, having just completed an intensive course on software modeling, CASE, and design patterns, completed an object-oriented software project in two versions: with no design patterns, and an "improved" version that employed a minimum of two design patterns. Results showed that, in almost all cases, the pattern versions required more lines of code to complete (13.5% more on average). [10]

These studies generally assessed patterns one at a time. Christensen argues that the benefit of design patterns is best realized when they are presented not as independent solutions to a specific problem but rather as part of a system of collaborating objects and systems, a point which is best communicated in practice rather than in theory. [11] Gestwicki and Sun similarly state that 'students are often unable to comprehend (design) patterns from isolated examples' even at the graduate level and that instead there is a need for 'software of significant scale' that contains multiple collaborative components to which design patterns can be applied. [19]

Khomh and Guéhéneuc surveyed professional software engineers and concluded that design patterns in fact have a negative impact on software quality. [20]

The study reports detailed statistics for 23 primitive design patterns across each of the three quality attributes. Some patterns, such as Composite and Prototype have universally positive ratings for each attribute. Most patterns, however, receive mixed ratings across the three categories. Most often, as with Adapter and Observer, patterns would receive positive ratings in one area - usually for expandability - and negative ratings in either understandability or reusability. Some patterns, such as Flyweight, received strongly negative ratings across the board.

The Visitor pattern, specifically, received a slightly positive rating for expandability but slightly negative for the other two. Because the Visitor pattern is generally considered to be one of the more difficult to comprehend and implement [28, 32], these generally neutral ratings are surprising, but the highly experienced nature of the respondents may account for this.

The authors conclude that patterns negatively impact several of the quality attributes assessed, and they "should be used with caution during development because they may actually impede maintenance and evolution." [20]

2.1.1 Teaching Patterns to Undergraduate Students. Beck, et al., made early claims that "Most people whom we have exposed to the concept of patterns can quickly become proficient at using the common ones." [7] Much work has since been done in bringing design patterns into the classroom. Many universities have created courses dedicated entirely to patterns, both at the graduate and undergraduate level, and some have endeavored to incorporate design patterns early and throughout foundational classes. [3, 36]

Others have attempted to introduce structure for the teaching of design patterns itself, or cataloging pedagogical patterns expressly designed for facilitating such teaching. [22]

Köppe lists several pitfalls in the teaching of patterns, and offers some potential strategies to avoid them:

- Students that start to learn patterns often hastily go straight to the solution and apply it.
- It's hard for students to see the advantages of correct applied patterns without direct experience.
- Students may try to show understanding by implementing as many patterns as possible.
- Students see patterns as something that intelligent people have written, not as "best practices," and that experienced developers use them without thinking about them. [22]

One of the fundamental problems of teaching design patterns in the classroom is that patterns are often introduced in a formulaic, almost sterile environment — introduced and described one after the other, as if specific patterns are merely items in a design catalog to be selected when needed. Missing is context — the application of the design pattern in one or more application domains or scenarios that are understandable by the student. [4, 30, 36]

Similarly, Stuurman and Florijn state that "design patterns are not learned by reading and doing small exercises." Instead, a "fairly large" program is necessary to provide a realistic context with which to build competence in identifying, applying, and combining design patterns. [31] Without context, "(t)eaching design patterns in isolation is similar to studying a foreign language with only a dictionary." [30]

Weiss extrapolates this idea further, suggesting that design patterns can be taught effectively to inexperienced undergraduates — in Weiss's case, freshmen programming students — by introducing patterns into examples without explicitly naming them as such. "Principles of good programming" — which happen to "invent" key design patterns can be introduced and, only after the value or benefit of the design is apparent, are design patterns pointed out and identified as such. Weiss calls this approach "teaching design patterns by stealth." [36]

Owen Astrachan agrees that care must be taken in the instruction of design patterns to inexperienced, undergraduate students and argues for a "simpler is better" approach and warns against what he calls "OO Overkill" and the unnecessary complication associated with incorporating design patterns in fundamental CS instruction. The problem arises, Astrachan argues, when educators focus too much attention on the inclusion of design patterns for their own sake and not enough on the situations and contexts in which those patterns are most appropriate. [3, 4]

Stuurman and Florijn wrote of similar experiences teaching design patterns to graduate students. At the conclusion of the course, many students reported that the three large assignments in the course were a significant factor that "helped them to grasp the concept of a certain design pattern: being confronted with a real problem was what they needed to 'see the light'." Astrachan concludes that design patterns and intensively object-oriented designs do not always scale down well from industrial applications to small classroom assignments. [31]

Cinnéide and Taylor seem to agree with the importance of context in presenting patterns to students, but further state that traditional classroom instruction is inadequate for pattern instruction. [12]

In a 2004 study, similar to an earlier experiments by Prechelt, they observed student performance in completing "functionally identical" programming exercises that either utilized patterns or omitted them. They observe that a survey indicates that students gained "a lot" from the experience and conclude that design patterns are "essential material to cover" in computer science curricula and suggest that "a non-traditional approach be taken to their presentation." [12]

3 STRUCTURE OF EXPERIMENT

Our experiment is guided by two questions: first, can young, inexperienced software engineers comprehend and apply design patterns; second, can the application of patterns by these engineers result in an increase in software quality and/or a reduction in development and refactoring costs.

To provide some insight into these questions, we conducted a small, controlled research trial of upperclass undergraduate computer science and software engineering students' performance on directed software tasks both with and without design patterns.

Statement	Choices	Percentage
What is the size, in lines of code, of the largest piece of software you have written or developed?	< 100	0.0%
	100 to 250	10.5%
	250 to 500	15.8%
	500 to 1,000	26.3%
	1,000 to 2,000	31.6%
	> 2,000	15.8%
Have you ever worked as a professional computer scientist or software engineer? If yes, indicate the number of years.	No	77.8%
	Part time	16.7%
	Full time	5.6%
	No. of years	1.25 (mean)
Please rate your knowledge of design patterns.	1-10 (low to high)	3.79 (mean)
Please rate your experience in working on software that utilizes design patterns.	1-10 (low to high)	3.32 (mean)

Table 1: Subject demographics and experience as self-reported in a pre-assessment survey.

3.1 Subject Demographics

Twenty subjects were recruited from among volunteers and were paid a nominal monetary fee for their participation. The average work experience of subjects was 0.28 years, with a maximum of 2 years of experience and a minimum of zero. Students self-reported that the largest previous software project upon which they had worked was, on average, around 1,000 lines of code. Similarly, students self-reported an average previous experience with design patterns of 3.32 on a 10-point Likert scale *(see Table 1)*. [23]

The subjects were surveyed to determine their experience both in writing software and in working with design patterns. Of those surveyed, the vast majority — 84.2% — indicated they had not previously written a piece of software in excess of 2,000 lines of code, with most subjects — 57.9% — reporting the largest piece of software they had written to be between 500 to 2,000 lines of code. Nearly 78% had no previous experience working as a professional developer, with only one respondent reporting previous full-time experience *(see Table 1)*.

Prior to participating in the experiment, subjects attended, as part of their regular classwork, two 75-minute lectures on design patterns. These lectures covered core design pattern topics including: an introduction to design patterns; simple patterns such as Singleton, Services, Service Providers and Locators, and the Null Object or Null Service pattern; and complex patterns including Observer, Composite, Visitor, and Object Pool.

Instruction was delivered in a lecture format and was supplemented with PowerPoint slides, code examples, and UML-style diagrams of relevant patterns.

3.2 Experiment Tasks

This experiment investigates the comprehension of design patterns through participants' actual study of code that implements patterns, with only some minor documentation in the code itself. This approach is intended to more closely resemble a real-world scenario in which a novice software engineer might be expected to analyze and

refactor code that he or she did not create and for which there may not be any existing models or formal documentation. This approach is a more rigorous and challenging assessment of design pattern capabilities than could be realized using a paper-based analysis of models or even printed code.

The experiment is comprised of three programming tasks, each of which focuses on one or more different design patterns, with limited overlap. The individual tasks used in this experiment are based upon software maintenance tasks originally designed and utilized by Prechelt, et al., in their paper-based evaluation of engineer performance with design patterns. [25] These tasks were later refactored and implemented in a C++ syntax by Vokáč in his computer-based study of the same topic. [35]

For this experiment, some of the Prechelt and Vokáč tasks have been refactored into a class-based Java implementation. Of the original tasks, the Boolean Formulas and Stock Ticker exercises have been retained essentially intact. In addition, a new task called Stock Service, which is loosely based on other tasks and incorporates the more modern and common Service and Service Provider patterns, has been added.

The Java language and jGRASP integrated development environment were selected because of the subjects' extensive undergraduate experience with both the language (on average, a minimum of three programming courses in the previous two years) and the environment in their curriculum. Additionally, the choice minimizes the potential for subjects to use non-standard code or platform-dependent API calls when refactoring each of the tasks. *(see Table 2)*

All subjects completed the experimental tasks in the same order, but were randomly assigned between the control and pattern-based implementations of each. Each subject completed either two non-pattern control tasks and one pattern-based task or vice versa. The distribution of subjects was such that an equal number of subjects were assigned to both the control and pattern implementations of each task. No two subjects sitting adjacent to each other worked on the same tasks at the same time.

3.3 Experiment Environment

On the dates of the experiments, students were asked during intake to complete a written survey prior to participating in the experiment. They were then randomly divided into groups for each of three pattern experiments. The experiments were administered in a quiet, moderately lit observation room with three computer terminals such that no more than three students worked on tasks at a time.

The experiment was conducted on Apple iMac computers that were provided to the students and pre-configured for the experiments. The code analysis and editing was accomplished using the jGRASP integrated development environment, with which all subjects had several semesters of previous development experience in computer science classes. Because the jGRASP environment is essentially identical in appearance regardless of the computing platform upon which it is installed, the use of iMac computers is not considered to be a factor in student performance.

Task start and end times were self-recorded by subjects using a digital clock that displayed the current time, including seconds.

	Classes		LOC	
Task	Control	Experiment	Control	Experiment
Boolean Formulas	9	11	299	366
Stock Ticker	9 *(+3 external)*	9 *(+3 external)*	261	336
Stock Service	5 *(+1 external)*	9 *(+1 external)*	234	381

Task	Patterns Employed
Boolean Formulas	Composite, Visitor
Stock Ticker	Observer
Stock Service	Service, Service Provider

Table 2: Experiment Tasks and the Design Patterns and Classes incorporated in each.

Following the three experiments, students were given a second written survey to complete before a verbal debriefing and dismissal.

3.4 Experiment Exercises

Boolean Formulas Exercise. The Boolean Formulas exercise is the most complex of the three employed in this study. The program is essentially a library for representing a boolean formula (using AND, OR, XOR, NOT and several variables) and for printing the formula in two different styles: infix notation on a single line, or prefix notation on multiple lines with indentation of nested terms.

A Boolean formula expressed in infix notation might be represented as follows:

```
((a XOR NOT b) AND (NOT x1 OR NOT x2))
```

The same formula expressed in prefix notation, with indentation, might be expressed in this way:

```
AND
  XOR
    A
    NOT
      B
  OR
    NOT
      x1
    NOT
      x2
```

The Boolean Formulas exercise is divided into two separate tasks, each of which requires the subject to implement a new printing routine. The first exercise task requires the subject to duplicate the single-line infix printing notation but with the actual value of terms (e.g. "true" or "false") instead of variable names. The second exercise task requires the subject print out the result of the evaluation of the entire formula, which again is either true or false.

The pattern implementation of this task consists of 11 classes comprising 366 lines of code. The formulas are represented using the Composite pattern and the printing of the formulas is implemented using the Visitor pattern. Because recursion is a necessary

component of the Composite pattern, and because the Visitor pattern is one of the more difficult to understand, this complexity adds to the learning time necessary to comprehend the implementation.

The control implementation consists of 9 classes comprising 299 lines of code. This implementation has the same Composite pattern as above, but is shorter and less complex. The printing is implemented without use of the Visitor pattern by a more straightforward method in each of the concrete classes. Adding new printing functionality requires adding methods to each of these classes.

To accomplish these tasks, the control group needed to implement a new printing method in each concrete class. In contrast, the patterns group needed to implement a new Visitor. As Vokáč hypothesized, "it should be easier to create a single new class similar to another existing class, rather than having to add methods to several classes" and this should provide an advantage to the patterns group. [34] The challenge in the patterns implementation, however, is overcoming the inherent complexity of understanding the Visitor pattern and then creating that new Visitor instance. If a subject does not grasp the Visitor implementation quickly, then the advantage of making a single change may not outweigh the simplicity of making multiple similar changes done by the control group.

Stock Ticker Exercise. The Stock Ticker Exercise consists of a program used for directing a continuous stream of stock trades, with information such as stock symbol, price, and volume, from a simulated stock market data service to one or more display windows.

The program consists of two or more classes that simulate streamed stock data from a stock market. A single stock market class provides a pre-defined sequence of simulated stock data that repeats continually. The stock market data is intended to be displayed by directing the data to one or more of pre-compiled display classes: one for the console, one with a single-line ticker window, and one with a multiline ticker window. Because the implementation of these classes and their drawing functions are not relevant to the exercise or the required task, subjects were not provided with the source code to these drawing classes. Instead, the compiled '.class' files were provided to permit subjects to use this functionality without the additional complexity of needing to understand their implementation.

The initial version of the program displays stock data in a single type of window. The subject's task is to enhance the program such that stock data is displayed simultaneously using all three of the provided display classes.

To accomplish this task, the subjects of both the control and patterns group must inspect and understand the implementation of the project. Both implementations are essentially identical except for the one class, called TradeInfoSource, that is responsible for obtaining the latest trade information from its source(s) and passing that information to the display(s) in use. In the control group, the retrieval and display of information is accomplished by the simple use of an infinite *while (true)* loop. In the patterns group, this class employs the Observer pattern, such that once a display class is "subscribed" to the TradeInfoSource's pattern, it and all other displays will automatically be provided with trade information without the need to modify the retrieval and display function of the class. The complexity of the Observer pattern implementation increases the length of the TradeInfoSource class by a factor of approximately four.

For the control group, the subject must realize that instances of the additional display classes must be declared and initialized. Next, control group subjects must realize that additional lines of code must be added to the TradeInfoSource class such that each time the class updates the existing display window it will also update the two additional display windows. An optimal solution for the control group can be accomplished by adding only four lines of code in two places.

For the patterns group, the subjects must not only inspect and comprehend the same utilization of the trade stream and display classes, but also understand the employment of the Observer pattern in the TradeInfoSource class. However, the use of the pattern class enables the optimal solution to the task to be accomplished in a single line of code for each new display that instantiates a new display object and then subscribes it to the available service.

Stock Service Exercise. The Stock Service exercise utilizes much of the same code employed in the previous Stock Ticker exercise, which subjects always complete first. However, in this implementation, the stock ticker display is connected to one of two possible simulated stock market services: one simulating data from the New York Stock Exchange, one with simulated data from the American Stock Exchange market. The goal of the exercise is to provide a capability to switch the ticker display between the two available market data streams or to "mute" the ticker display entirely. The choice can be changed repeatedly until the user elects to quit the program.

The control group implementation of this program provides a simple console menu that offers the user a choice between displaying a single available market stream or exiting the program. The correct implementation of the program requires the subject to merely modify to menu to add additional choices and duplicate the *if* statement used to open and display the selected stream.

The patterns group implementation adds the complexity of the Service Provider pattern and includes an abstract StockService class, two concrete Service Provider classes for both market services, and an additional concrete EmptyMarketServiceProvider class that provides an empty stream of data. Finally, an additional StockServiceLocator class is provided that allows registration of an available stock service and provides that service to the stock ticker class when needed. The complexity of the Service Provider class removes one class, adds four new concrete classes and one abstract class, and an additional 163 lines of code (a 65% increase).

4 EXPERIMENTAL RESULTS

Subjects self-reported start and end times, and we computed the difference between the start and end time as the elapsed work time. Subjects took no breaks and had no interruptions that would have skewed reported times. The precision of measurement was one second, with an assumed accuracy of ±10 seconds.

Stock Ticker Exercise. The mean time required for completion by subjects in the control group was 997.1 seconds, with a standard deviation of 833.6 seconds. In comparison, the mean time required

by subjects in the control group was 808.6 seconds, with a standard deviation of 415.8 seconds. A one-tailed test (left-tailed test), such that $time(H_1) < time(H_0)$, produced a p-value of 0.2666. [33] Outlier checking consisted of determining quartiles and then examining each data point in relation to those quartiles. The second quartile (Q_2) is the median. The first quartile (Q_1) is the median of lower half of all measurements; the third quartile (Q_3) is the median of the upper half of all measurements. We then determined the interquartile range, which is the range of the middle 50 percent of samples, or $Q_3 - Q_1$, and computed the upper and lower fence, which serve as cutoff points for determining outliers.

$$Fence_{lower} = Q_1 - 1.5(IQR), \quad Fence_{upper} = Q_3 + 1.5(IQR)$$

There are no outliers on the lower scale for either the patterns or control group, but the upper fence value was exceeded by one subject, which reported a time of 3180, and we classified the data from that subject A1 as a high outlier. When this outlier is excluded, the control group's mean becomes 754.6 seconds and the standard deviation becomes 346.3 seconds. If we then perform a one-tailed t-test, this produces a higher p-value of 0.3806.

We see that performance in the Stock Ticker exercise shows that the control group performs nearly as well as the patterns group. Although the data indicate slightly better median and mean performance by the patterns group, there is a non-negligible probability that these data are not random and these results cannot be proven statistically significant. The data do not show a decrease in performance for the patterns group beyond a traditional certainty of statistical significance.

The mean time required for completion by subjects in the design patterns group was 611.0 seconds, with a standard deviation of 182.1 seconds. In comparison, the mean time required by subjects in the control group was 1130.7 seconds, with a standard deviation of 633.3 seconds. Performing a one-tailed t-test produces a p-value of 0.0207.

Stock Service Exercise. The mean time required for completion by subjects in the design patterns group was 611.0 seconds, with a standard deviation of 182.1 seconds. In comparison, the mean time required by subjects in the control group was 1130.7 seconds, with a standard deviation of 633.3 seconds. Performing a one-tailed t-test produces a p-value of 0.0207.

Checking again for outliers, we see that there are no outliers in the patterns group. For the control group, there are no outliers on the lower end of the scale, but one subject in the control group reported a time of 2693, significantly above the upper fence, and this data point was classified as an outlier. If the outlier in the control group is excluded, statistical analysis shows more confidence in the data, with the control group's mean becoming 935.4 and the standard deviation 257.1. Again performing a one-tailed t-test produces a small p-value of 0.0056, indicating a strong probability that the data are not random.

We see that the patterns group vastly outperform the control group in the Stock Service exercise. The median and mean of the patterns group are roughly half that of the control group. Indeed, nearly all of the patterns group perform better than the top three quartiles of the control group. There is a strong statistical significance to the results and we conclude that — absent fundamental error in the structure of this exercise — the data support our hypothesis in this case.

Boolean Formulas Exercise. If we inspect the data as before, we see there are no outliers for either task, or the combined elapsed time for both tasks, in either group. Because the Boolean Formulas exercise was comprised of two sequential tasks, we will look subject performance in each separately and then the cumulative performance over both tasks.

For the first task, the mean time required for completion by subjects in the design patterns group was 3065.1 seconds, with a standard deviation of 1896.9 seconds. In comparison, the mean time required by subjects in the control group was 2786.7 seconds, with a standard deviation of 1360.1 seconds. A two-tailed t-test produces a t-value of 0.362985 and a p-value of 0.721367. A one-tailed t-test produces a p-value of 0.4667.

For the second task, the mean time required for completion by subjects in the design patterns group was 2852.6 seconds, with a standard deviation of 1257.83 seconds. In comparison, the mean time required by subjects in the control group was 2049.4 seconds, with a standard deviation of 2004.0 seconds. A two-tailed t-test produces a t-value of 0.9344 and a p-value of 0.3649. A one-tailed t-test produces a p-value of 0.1634.

For the cumulative time to execute both tasks, the mean time required for completion by subjects in the design patterns group was 5571.7 seconds, with a standard deviation of 1702.2 seconds. In comparison, the mean time required by subjects in the control group was 4836.1 seconds, with a standard deviation of 1635.1 seconds. A two-tailed t-test produces a t-value of .8979 and a p-value of 0.3834. A one-tailed t-test produces a p-value of 0.1947.

4.1 Threats to Validity

Based on lessons learned in others' research, controls were put in place to minimize threats to the validity of experiment data. [24] Subjects in the experiment seemed willing to perform as best they could. In fact, outlier students who struggled with tasks labored extensively (in some cases up to 120 minutes) before they were willing to give up on the assigned task or tasks. Environmental conditions were controlled and identical for all subjects, and subjects were provided a quiet workplace free from any external distractions. The design of the experiment included counter-balancing to reduce or control for differences in experience, learning, and ability of subjects. Experiment data was self-collected by subjects on paper and then verified for accuracy by the researcher collecting the completed paperwork.

Additionally, we controlled for a possible "learning effect" among subjects. That is, we wished to avoid bias in performance times due to similarities in source code structure in different tasks. For example, the stock ticker and stock service tasks share similar classes and components. Were some subjects to see one task before the other, they could be reasonable be expect to require less time to become familiar with the code in the latter task. To prevent this, all tasks were presented to all subjects in the same order - regardless of whether any specific subject was in a control or experimental group for any given task.

The most significant threats are to external validity, specifically factors that may limit the ability to extrapolate findings beyond

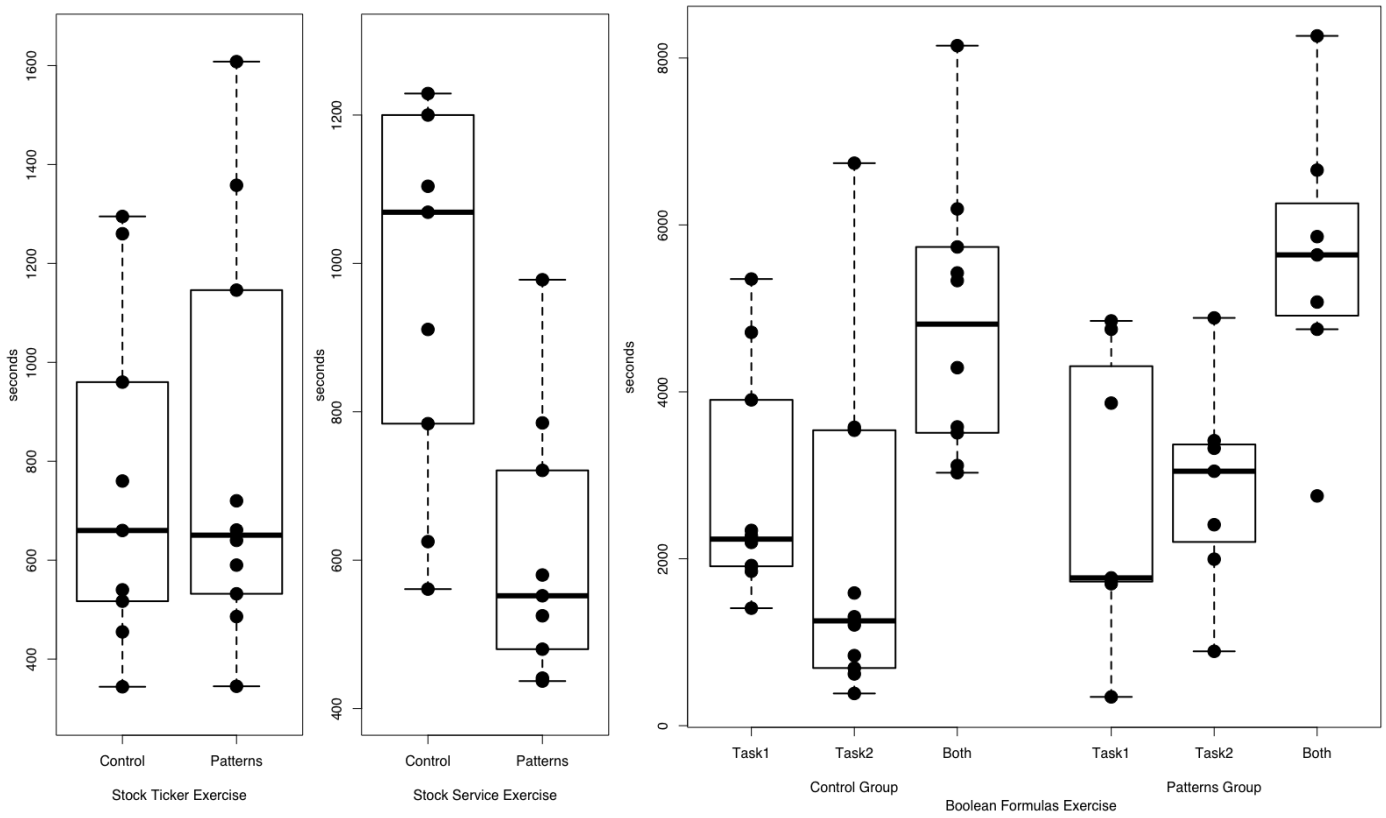

Figure 1: Comparison of Control and Patterns group completion times for the Stock Ticker, left, Stock Service, center, and Boolean Formulas, right, exercises.

the population and environment studied. First of these factors is instructor quality; that is, the adequate communication of knowledge and lesson content to students. Second is subject experience; that is, if we presume that design pattern performance is either loosely or tightly coupled to a software engineer's experience, then students' background plays a significant role. [3]

The problems and source code examples involved in this experiment are relatively small, and do not reflect the size of typical real-world projects, either in terms of lines of code or the number of interacting classes and objects. Prechelt suggests that this fact does not reduce the validity of any results in experiments of this type, but rather that larger and more complex projects may in fact realize greater benefits from pattern use. [24]

We accept as a given that much of the time required in completing these tasks was spent in analyzing and understanding the source code, class interaction, and pattern implementation (when present). While the data cannot predict whether performance on other types of tasks would be similar, we suggest these results can be interpreted to mean that tasks of higher complexity involving similar subjects will benefit as much or more from pattern adoption.

Factors related to programming experience are mitigated because nearly all subjects have similar levels of software development experience.

Statement	Available Choices	Pre-Test	Post-Test
Please rate your knowledge of design patterns.	1-10 Scale (low-high)	3.79 (mean)	5.56 (mean)
Please rate your experience in working on software that utilizes design patterns.	1-10 Scale (low-high)	3.32 (mean)	4.94 (mean)

Table 3: Pre- and Post-Assessment survey of experience and design pattern familiarity.

5 SUBJECTIVE SURVEY RESULTS

Subjects in this study were given a pre- and post-assessment survey that measured both quantitative data regarding the subjects' experience in both software development and design patterns, subjective opinions on familiarity with relevant terms, and subjective opinions on the suitability and ease of use of design patterns. Additionally, after completing each exercise in the study, subjects were asked a number of subjective questions relevant to that particular exercise.

Term	Never heard of it	Heard of it, but not sure	Know a little about it	Know a lot about it
Design Patterns	0%	31.6%	63.2%	5.26%
Singleton	10.5%	57.9%	31.6%	0%
Service Locators and Service Providers	26.3%	47.3%	26.3%	0%
Observer Pattern	57.9%	42.1%	0%	0%
Object Pool or Resource Pool	36.8%	47.4%	15.8%	0%
Visitor Pattern	79.0%	21.0%	0%	0%
Iterator Pattern	52.6%	31.6%	15.8%	0%

Table 4: Pre-Assessment survey of subject familiarity with relevant terms.

5.1 Pre- and Post-Assessment Questionnaires

Prior to taking the assessment, subjects in the study rated themselves on average a 3.79, on a scale from 1 to 10, in response to the statement "Please rate your knowledge of design patterns." However, at the conclusion of the assessment, subjects rated themselves an average of 5.56 to the same question, indicating that the subjects felt their knowledge of design patterns increased by participating in the study *(see Table 3)*.

Similarly, prior to taking the assessment, subjects rated themselves an average of 3.32 in response to the statement "Please rate your experience in working on software that utilizes design patterns." Subjects' rating of this statement also increased after the assessment to an average of 4.94.

Both before and after the assessment, subject rated their knowledge about specific design patterns terms. Each subject was also asked to respond to each of the following questions:

(1) Do you feel that implementing software using design patterns is difficult?
(2) Do you feel that implementing software using design patterns takes more or less time than software without design patterns?
(3) How likely are you to attempt to utilize design patterns in software that you develop?

Subjects were also given a list of selected terms related to design patterns and asked to rate their familiarity with the terms on a four-choice scale *(see Table 4)*. For each term, subjects selected one of the following: Never heard of it; Heard of it, but not sure; Know a little about it; Know a lot about it.

With a single exception, no subjects selected "Know a lot about it" for any term. The vast majority of subjects indicated only a cursory familiarity with terms, selecting "Heard of it, but not sure" roughly half the time. For some terms, such as the Visitor and Iterator patterns, the majority of subjects selected "Never heard of it".

Prior to the assessment, 46.2% of subjects felt that implementing design patterns was either very difficult or slightly difficult. At the conclusion of the survey, that percentage was essentially unchanged at 46.1%, but the data still show an overall upward shift, with the percentage of those choosing "very difficult" decreasing and those choosing "slightly difficult" increasing. Interestingly, subjects who selected "very easy" in prior to the assessment all reduced their ratings to "slightly easy" at the conclusion of the assessment *(see Table 5)*.

Statement	Available Choices	Pre-Test	Post-Test
Do you feel that implementing software using design patterns is difficult?	Don't know	0.0%	0.0%
	Very difficult	7.7%	5.9%
	Slightly difficult	38.5%	41.2%
	Slightly easy	46.1%	52.9%
	Very easy	7.7%	0.0%
Do you feel that implementing software using design patterns takes more or less time time than software without design patterns?	Don't know	0.0%	0.0%
	Much less time	45.5%	18.8%
	Slightly less time	27.2%	25.0%
	Slightly more time	18.2%	43.8%
	Much more time	9.1%	12.5%
How likely are you to attempt to utilize design patterns in software that you develop?	Very unlikely	5.3%	0.0%
	Somewhat unlikely	5.3%	5.9%
	Somewhat likely	42.1%	35.3%
	Very likely	47.4%	58.8%

Table 5: Pre- and Post-Assessment survey of subjective opinions regarding design patterns.

Subjects similarly shifted their opinions on the amount of time required to implement patterns. Prior to the assessment, 73.7% of subjects felt that implementing software using design patterns would require "much less time" or "slightly less time" than software without design patterns. Of these, nearly half of all subjects - 45.5% - thought it would take "much less time." However, after completing the assessment, only 43.8% felt the same, with only 18.8% still feeling that design patterns take "much less time" *(see Table 5)*.

This reveals a very large shift in opinion during the course of the assessment. This change might be related to the inexperience of the subjects with actual design pattern usage. Since the subjects had received classroom instruction on patterns - instruction that arguably was intended to present the advantages of using design patterns - they could reasonably have expected software using patterns to be an all-around improvement over non-pattern software. When seeing and working with patterns for the first time, subjects unsurprisingly required time to identify and understand patterns in exercise code. This learning curve may have been steeper than subjects expected, which would account for these changing pre- and post-assessment opinions.

A similar revision of opinions is seen in responses to the question "How likely are you to attempt to utilize design patterns in software that you develop?" Prior to the assessment, almost all subjects - 89.5% answered "somewhat likely" or "very likely", with nearly half - 47.4% - choosing "very likely". After the assessment, however, the percentage indicating they were likely to utilize patterns increased slightly to 94.1%, but the percentage choosing "very likely" increased significantly to 58.5% *(see Table 5)*.

6 CONCLUSIONS

Although the data are indeterminate in one of the three trials, we find strong evidence that novice software engineers can recognize certain design patterns in software and refactor that software quickly to accomplish a desired task. In other cases, we note that the inclusion of design patterns can vastly increase the amount of time needed for novice software engineers to understand and refactor software containing such patterns. Although this experiment only consisted of four refactoring tasks employing only a handful of design patterns, we note the circumstantial evidence that, the simpler the design pattern, the more likely inexperienced software engineers are to recognize the pattern and refactor it correctly and efficiently. We suggest, but cannot prove, that the more complex design patterns, such as the Visitor pattern, are likely to degrade the performance of software engineers with limited experience in writing software and working with patterns.

REFERENCES

[1] Christopher Alexander, Sara Ishikawa, and Murray Silverstein. 1977. *A Pattern Language: Towns, Buildings, Construction.* Oxford University Press. 1171 pages.

[2] Association for Computing Machinery 1994. Why a Conference on Pattern Languages? *Software Engineering Notes* 19 (January 1994), 50-52 pages.

[3] Owen Astrachan. [n. d.]. Presentation: OO Overkill - When Simple is Better than Not. ([n. d.]). PowerPoint Presentation provided by Duke University Department of Computer Science.

[4] Owen Astrachan. 2001. OO Overkill: when simple is better than not. (2001), 302-306 pages. https://doi.org/10.1145/364447.364608

[5] Kent Beck. 1997. *Smalltalk Best Practice Patterns.* Prentice Hall.

[6] K. Beck. 2008. *Implementation Patterns.* Addison-Wesley. http://books.google.com/books?id=i40hAQAAIAAJ

[7] Kent Beck, Ron Crocker, Gerard Meszaros, John Vlissides, James O. Coplien, Lutz Dominick, and Frances Paulisch. 1996. Industrial Experience with Design Patterns. In *Proceedings of the 18th International Conference on Software Engineering (ICSE '96).* IEEE Computer Society, Washington, DC, USA, 103–114. http://dl.acm.org/citation.cfm?id=227726.227747

[8] Kent Beck and Ward Cunningham. 1987. *Using Pattern Languages for Object-Oriented Programs.* Technical Report.

[9] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal. 1996. *Pattern-Oriented Software Architecture, A System of Patterns.* John Wiley and Sons Ltd., West Sussex PO19 8SQ, England. http://books.google.com/books?id=j_ahu_BS3hAC

[10] Alexander Chatzigeorgiou, Nikolaos Tsantalis, and Ignatios Deligiannis. 2008. An Empirical Study on Students' Ability to Comprehend Design Patterns. *Comput. Educ.* 51, 3 (Nov. 2008), 1007–1016. https://doi.org/10.1016/j.compedu.2007.10.003

[11] Henrik Baerbak Christensen. 2004. Frameworks: putting design patterns into perspective. (2004), 142-145 pages. https://doi.org/10.1145/1007996.1008035

[12] Mel Cinneide and Richard Tynan. 2004. A problem-based approach to teaching design patterns. *SIGCSE Bull.* 36, 4 (2004), 80–82. https://doi.org/10.1145/1041624.1041663

[13] James O. Coplien. 1992. *Advanced C++ Programming Styles and Idioms.* Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA.

[14] James O. Coplien. 1994. Progress on Patterns: Highlights of PLoP/94. In *Proceedings of Object Expo Europe.*

[15] Erich Gamma. 2002. *Design patterns: ten years later.* Springer-Verlag New York, Inc., 688–700.

[16] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. 1993. Design patterns: Abstraction and reuse of object-oriented design. In *ECOOP '93.* Springer-Verlag, 406–431.

[17] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. 1995. *Design patterns: elements of reusable object-oriented software.* Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA.

[18] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. 2002. *Design patterns: abstraction and reuse of object-oriented design.* Springer-Verlag New York, Inc., 701–717.

[19] Paul Gestwicki and Fu-Shing Sun. 2008. Teaching Design Patterns Through Computer Game Development. *J. Educ. Resour. Comput.* 8, 1 (2008), 1–22. https://doi.org/10.1145/1348713.1348715

[20] F. Khomh and Y. G. Gueheneuc. [n. d.]. Do Design Patterns Impact Software Quality Positively?. In *Software Maintenance and Reengineering, 2008. CSMR 2008. 12th European Conference on.* 274–278. 10.1109/CSMR.2008.4493325

[21] Barbara Kitchenham, O. Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic Literature Reviews in Software Engineering - A Systematic Literature Review. *Inf. Softw. Technol.* 51, 1 (Jan. 2009), 7–15. https://doi.org/10.1016/j.infsof.2008.09.009

[22] Christian Köppe. 2011. A Pattern Language for Teaching Design Patterns (Part 2). In *Proceedings of the 18th Conference on Pattern Languages of Programs (PLoP '11).* ACM, New York, NY, USA, Article 23, 16 pages. https://doi.org/10.1145/2578903.2579161

[23] Saul McLeod. 2008. The Likert Scale. (2008). http://www.simplypsychology.org/likert-scale.html

[24] Lutz Prechelt. 1997. *An experiment on the usefulness of design patterns: Detailed description and evaluation.* Report. https://doi.org/TechnicalReport9/1997

[25] Lutz Prechelt and Barbara Unger. 1998. A Series of Controlled Experiments on Design Patterns: Methodology and Results. (1998), 53–60 pages.

[26] L. Prechelt and B. Unger. 2001. An experiment measuring the effects of personal software process (PSP) training. *Software Engineering, IEEE Transactions on* 27, 5 (2001), 465–472.

[27] Lutz Prechelt, Barbara Unger, and Michael Philippsen. 1997. Documenting design patterns in code eases program maintenance. (1997), 72–76 pages.

[28] L. Prechelt, B. Unger, W. F. Tichy, P. Brossler, and L. G. Votta. 2001. A controlled experiment in maintenance: comparing design patterns to simpler solutions. *Software Engineering, IEEE Transactions on* 27, 12 (2001), 1134–1144. 10.1109/32.988711

[29] L. Prechelt, B. Unger-Lamprecht, M. Philippsen, and W. F. Tichy. 2002. Two controlled experiments assessing the usefulness of design pattern documentation in program maintenance. *Software Engineering, IEEE Transactions on* 28, 6 (2002), 595–606. 10.1109/TSE.2002.1010061

[30] Asher Sterkin. 2006. Teaching Design Patterns. In *FECS*, Hamid R. Arabnia (Ed.). CSREA Press, 167–176.

[31] Sylvia Stuurman and Gert Florijn. 2004. Experiences with Teaching Design Patterns. *SIGCSE Bull.* 36, 3 (June 2004), 151–155. https://doi.org/10.1145/1026487.1008037

[32] James Sugrue. 2012. Design Patterns Uncovered: The Visitor Pattern. (March 2012). http://java.dzone.com/articles/design-patterns-visitor

[33] Michael Sullivan III. 2013. *Statistics: Informed Decisions Using Data.* Peason Education, Inc.

[34] Marek Vokáč. 2004. *On the Practical Use of Software Design Patterns.* Ph.D. Dissertation. University of Oslo.

[35] Marek Vokáč, Walter Tichy, DagI.K. SjØberg, Erik Arisholm, and Magne Aldrin. 2004. A Controlled Experiment Comparing the Maintainability of Programs Designed with and without Design Patterns—A Replication in a Real Programming Environment. *Empirical Software Engineering* 9, 3 (2004), 149–195. https://doi.org/10.1023/B:EMSE.0000027778.69251.1f

[36] Stephen Weiss. 2005. Teaching Design Patterns by Stealth. In *Proceedings of the 36th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '05).* ACM, New York, NY, USA, 492–494. https://doi.org/10.1145/1047344.1047500

[37] P. Wendorff. [n. d.]. Assessment of design patterns during software reengineering: lessons learned from a large commercial project. In *Software Maintenance and Reengineering, 2001. Fifth European Conference on.* 77–84.

[38] Cheng Zhang and D. Budgen. 2012. What Do We Know about the Effectiveness of Software Design Patterns? *Software Engineering, IEEE Transactions on* 38, 5 (Sept 2012), 1213–1231. https://doi.org/10.1109/TSE.2011.79